Python Cheatsheet

Keywords explained · Beginner to Professional · Python 3.10+ (newer features are marked)

BEGINNER syntax, types, control flow, functions | **INTERMEDIATE** comprehensions, files, OOP, modules | **ADVANCED** generators, decorators, typing, stdlib | **PROFESSIONAL** async, testing, tooling, performance

A. The 35 reserved keywords (Lv = level at which you normally learn it)

Keyword	Lv	What it does	Example
`False`	B	Boolean false value (falsy).	`ok = False`
`None`	B	The 'no value' singleton. Test with `is None`.	`if x is None: ...`
`True`	B	Boolean true value.	`done = True`
`and`	B	Logical AND; stops at the first falsy operand.	`if a and b: ...`
`as`	B	Gives an alias in import, with and except.	`import numpy as np`
`assert`	I	Raises AssertionError if condition is false (debug/tests).	`assert n > 0, 'n must be > 0'`
`async`	P	Declares a coroutine; also async with / async for.	`async def fetch(): ...`
`await`	P	Suspends a coroutine until an awaitable finishes.	`data = await fetch()`
`break`	B	Leaves the nearest enclosing loop immediately.	`if x == 3: break`
`class`	I	Defines a class (blueprint for objects).	`class Dog(Animal): ...`
`continue`	B	Skips to the next loop iteration.	`if x % 2: continue`
`def`	B	Defines a function.	`def add(a, b): return a + b`
`del`	B	Deletes a name, list item, slice or dict key.	`del d['key']`
`elif`	B	Else-if: another condition after if.	`elif x == 0: ...`
`else`	B	Fallback branch; also after loop (no break) / try (no error).	`else: print('none')`
`except`	B	Catches exceptions raised inside try.	`except ValueError as e: ...`
`finally`	B	Always runs, even after errors; used for cleanup.	`finally: f.close()`
`for`	B	Iterates over any iterable.	`for i in range(3): ...`
`from`	B	Imports names; chains errors; delegates generators.	`from os import path`
`global`	I	Lets a function rebind a module-level variable.	`global counter`
`if`	B	Runs a block when the condition is truthy.	`if x > 0: ...`
`import`	B	Loads a module into the namespace.	`import json`
`in`	B	Membership test; also the loop iterator source.	`'a' in 'cat'`
`is`	B	Identity test: same object? (not equality).	`x is None`
`lambda`	I	Anonymous one-expression function.	`lambda x: x * 2`
`nonlocal`	A	Rebinds a variable in the enclosing function scope.	`nonlocal total`
`not`	B	Logical negation.	`if not items: ...`
`or`	B	Logical OR; stops at the first truthy operand.	`name = a or 'default'`
`pass`	B	Does nothing; syntactic placeholder.	`class Stub: pass`
`raise`	I	Throws an exception (bare raise re-throws).	`raise ValueError('bad')`
`return`	B	Exits a function and sends back a value.	`return result`
`try`	B	Starts a block whose exceptions can be handled.	`try: risky() ...`
`while`	B	Repeats while the condition is truthy.	`while n > 0: n -= 1`
`with`	I	Runs a context manager; guarantees cleanup.	`with open(p) as f: ...`
`yield`	A	Turns a function into a generator; produces lazily.	`yield item`

B. Soft keywords (only special in a particular context)

Keyword	Lv	What it does	Example
`match / case`	I	Structural pattern matching (3.10+). Only keywords inside a match block.	`match cmd: case 'go': ...`
`type`	A	Type-alias statement (3.12+). Still usable as a normal name.	`type Vec = list[float]`
`_`	B	Throwaway variable / wildcard pattern / last REPL result.	`for _ in range(3): ...`

Tip: inside the REPL use `help(obj)`, `dir(obj)` and `import this` (the Zen of Python). All 35 keywords: `import keyword; keyword.kwlist`.

C. Built-in functions & types you will use constantly

`abs(x)`	absolute value	`isinstance(x, T)`	type check (incl. subclasses)
`all(it)`	True if all truthy	`issubclass(A, B)`	class relation check
`any(it)`	True if any truthy	`iter(x) / next(it)`	manual iteration
`bin / hex / oct(n)`	number as base-2/16/8 string	`len(x)`	number of items
`bool(x)`	convert to bool	`map(f, it)`	apply f to each item
`breakpoint()`	drop into debugger (pdb)	`max / min(it, key=f)`	largest / smallest
`bytes / bytearray`	binary data	`memoryview(b)`	zero-copy buffer view
`callable(x)`	can x be called?	`object()`	base of all classes
`chr(n) / ord(c)`	code point <-> character	`open(path, mode)`	open a file
`classmethod / staticmethod`	method decorators	`pow(a, b, mod)`	power (optional modulo)
`dict / list / set / tuple`	build collections	`print(*a, sep, end)`	write to stdout
`dir(x)`	list attributes	`property(...)`	managed attribute
`divmod(a, b)`	(quotient, remainder)	`range(a, b, step)`	integer sequence
`enumerate(it, start)`	(index, item) pairs	`repr(x)`	developer string of x
`eval / exec`	run code strings (avoid!)	`reversed(seq)`	reverse iterator
`filter(f, it)`	keep items where f is true	`round(x, n)`	round (banker's rounding)
`float / int / str`	type conversion	`slice(a, b, c)`	slice object
`format(x, spec)`	format a value	`sorted(it, key=f)`	new sorted list
`frozenset(it)`	immutable set	`sum(it, start)`	add up items
`getattr / setattr / hasattr`	dynamic attribute access	`super()`	access the parent class
`globals() / locals()`	namespace dicts	`type(x)`	type of x / make a class
`hash(x)`	hash value	`vars(x)`	__dict__ of x
`help(x)`	built-in documentation	`zip(a, b)`	pair items in parallel
`id(x)`	object identity number	`aiter / anext`	async iteration
`input(prompt)`	read a line (always str)	`__import__(n)`	low-level import

D. Syntax at a glance

Literals & escapes

`0b101 0o17 0xFF`	binary, octal, hex integers	`f'{x}'`	formatted string literal
`1_000_000`	underscores for readability	`'''text'''`	multi-line string
`1e3 3.14`	float literals	`\n \t \\ \'`	newline, tab, backslash, quote
`3+4j`	complex number	`\u00e9`	Unicode escape
`r'\d+'`	raw string (backslashes kept)	`...`	Ellipsis: placeholder / numpy slicing
`b'abc'`	bytes literal	`None True False`	singleton constants

Format spec: `f'{value:spec}'` or `format(value, spec)`

`:.2f`	2 decimals (3.14)	`:e`	scientific notation
`:>8 :<8 :^8`	right / left / centre, width 8	`{x!r}`	use repr() of x
`:08.3f`	zero-padded	`{x=}`	debug print: x=5
`:,`	thousands separator (1,234)	`:*^20`	fill char + alignment
`:.1%`	percent (12.5%)	`:%Y-%m-%d`	datetime format
`:b :x :o`	binary / hex / octal	`:>{w}`	nested width from variable

Operator precedence (highest to lowest)

`(...)`	grouping (highest)	`& ^ |`	bitwise AND, XOR, OR
`x[i] x.attr f()`	subscript, attribute, call	`== != < > in is`	comparison, membership, identity
`**`	power	`not`	boolean NOT
`+x -x ~x`	unary	`and`	boolean AND
`* / // % @`	multiply-like	`or`	boolean OR
`+ -`	add, subtract	`x if c else y`	ternary
`<< >>`	bit shifts	`:=`	walrus (lowest)

LEVEL 1 · BEGINNER

Learn the syntax and think in values, types and control flow.

Variables & core types

`x = 10`	Assign; no declaration needed, the type is dynamic.
`int float str bool`	Whole number, decimal, text, True/False.
`type(x) isinstance(x, int)`	Inspect type; isinstance accepts subclasses too.
`int('5') str(5) float('2.5')`	Convert (cast) between types.
`a, b = 1, 2`	Unpacking. Swap with `a, b = b, a`.
`None`	Represents 'nothing'; compare using `is None`.
`# note '''doc'''`	Comment; triple-quoted docstring documents code.
`snake_case PascalCase UPPER`	Naming: variables/functions, classes, constants.

Operators

`+ - * / // % **`	add, subtract, multiply, true-divide, floor-divide, modulo, power.
`== != < > <= >=`	Comparisons; can chain: `0 < x < 10`.
`and or not`	Boolean logic (short-circuit).
`in not in`	Membership: `'a' in 'cat'`.
`is is not`	Identity (same object); use for `None`.
`+= -= *= //=`	Augmented assignment.
`:=`	Walrus: assign inside an expression, `if (n := len(a)) > 3:`.
`& | ^ ~ << >>`	Bitwise AND, OR, XOR, NOT, shifts.

Falsy values: `0 0.0 '' [] {} set() None False`. Everything else is truthy.

Strings (immutable)

`s[0] s[-1] s[1:4] s[::-1]`	First, last, slice, reversed copy.
`f'{name}: {n:>8.2f}'`	f-string with format spec (width, precision).
`s.upper() .lower() .title()`	Change case.
`s.strip() .lstrip() .rstrip()`	Trim whitespace/characters.
`s.split(',') ','.join(lst)`	Split into list; join list into string.
`s.replace(a, b)`	Replace all occurrences.
`s.find(x) s.index(x)`	Position; find gives -1, index raises ValueError.
`s.startswith(p) .endswith(p)`	Prefix / suffix test.
`s.isdigit() .isalpha()`	Character-class checks.
`len(s)`	Number of characters.

Control flow

```
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")

for i in range(3):          # 0, 1, 2
    print(i)

while n > 0:
    n -= 1
    if n == 5: break        # leave loop
    if n % 2: continue      # next iteration

label = "yes" if ok else "no"     # ternary
```

Collections

`[1, 2, 3] list`	Ordered, mutable, allows duplicates.
`(1, 2) tuple`	Ordered, immutable; `(1,)` is one item.
`{'a': 1} dict`	Key to value map; keeps insertion order.
`{1, 2} set`	Unique items; very fast membership test.
`lst.append(x) .extend(it)`	Add one item / many items.
`lst.insert(i, x) .remove(x)`	Insert at index / remove first match.
`lst.pop(i) .sort() .reverse()`	Remove & return; sort / reverse in place.
`lst[::2] lst[:]`	Every 2nd item; shallow copy.
`d.get(k, default)`	Safe lookup (no KeyError).
`d.keys() .values() .items()`	Views to loop over.
`d.update(o) d | o`	Merge dicts (`|` needs 3.9+).
`d.setdefault(k, v) d.pop(k)`	Insert if missing; remove key.
`s.add(x) s.discard(x)`	Set add / remove (no error if absent).
`a | b a & b a - b a ^ b`	Union, intersection, difference, symmetric diff.

Functions

```
def greet(name, greeting="Hello"):
    """Return a greeting string."""
    return f"{greeting}, {name}!"

greet("Ana")                        # positional
greet(greeting="Hi", name="Bo")     # keyword arguments
```

No `return` means `None` is returned. Default values are evaluated once, at definition time: never use `[]` or `{}` as a default.

Input / output, loops & errors

`print(a, b, sep=', ', end='')`	Output; sep goes between, end goes after.
`input('Age? ')`	Reads a line; always returns `str`.
`range(5) range(2, 10, 2)`	0..4; 2, 4, 6, 8.
`enumerate(lst, 1)`	Index + item pairs, starting at 1.
`for k, v in d.items():`	Loop over dict pairs.

```
try:
    n = int(input("n: "))
except ValueError as e:
    print("Not a number:", e)
else:
    print("ok")         # runs if no exception
finally:
    print("done")       # always runs
```

Common exceptions: ValueError, TypeError, KeyError, IndexError, AttributeError, ZeroDivisionError, FileNotFoundError, ImportError.

LEVEL 2 · INTERMEDIATE

Write idiomatic, reusable code: comprehensions, files, modules, classes.

Comprehensions

```
[x * x for x in range(10) if x % 2 == 0]   # list
{k: v for k, v in pairs}                    # dict
{x % 3 for x in nums}                       # set
(x * x for x in nums)                       # generator
[y for row in grid for y in row]            # flatten
```

Functional helpers

`lambda x: x * 2`	Anonymous one-expression function.
`sorted(p, key=lambda t: t[1])`	Sort by key function (stable sort).
`map(f, it) filter(f, it)`	Lazy transform / keep truthy results.
`zip(a, b) zip(*rows)`	Pair up / transpose. `strict=True` checks lengths.
`any(it) all(it)`	At least one / every item truthy.
`sum(it) min(it) max(it, key=f)`	Aggregate.

Arguments & scope

```
def f(a, b=2, *args, key=None, **kwargs):
    ...                       # args: tuple, kwargs: dict

def g(pos_only, /, normal, *, kw_only): ...

f(*[1, 2, 3], **{"key": 1})  # unpack when calling
```

Scope lookup order (LEGB): Local, Enclosing, Global, Built-in. Use `global x` / `nonlocal x` to rebind outer names.

Modules & the standard library

`import m from m import x as y`	Import a module / a name with alias.
`if __name__ == '__main__':`	Run only when executed as a script.
`pkg/__init__.py`	A folder with this file is a package.
`os sys pathlib`	Operating system, interpreter, file paths.
`json csv re`	Data formats and regular expressions.
`datetime time random math`	Dates, time, randomness, maths.
`argparse logging subprocess`	CLI args, logs, running programs.

Files & JSON

```
with open("in.txt", encoding="utf-8") as f:
    for line in f:                  # lazy, line by line
        print(line.rstrip())

with open("out.txt", "w", encoding="utf-8") as f:
    f.write("hello\n")

import json
data = json.loads('{"a": 1}')       # str  -> dict
text = json.dumps(data, indent=2)   # dict -> str
```

Modes: `r` read, `w` overwrite, `a` append, `x` create-only, `b` binary, `+` read/write. Always pass `encoding=`.

Classes & objects

```
class Dog(Animal):                  # inheritance
    species = "canis"               # class attribute

    def __init__(self, name):       # constructor
        super().__init__()
        self.name = name            # instance attribute

    def speak(self):
        return f"{self.name}: woof"

    def __repr__(self):
        return f"Dog({self.name!r})"
```

`@property`	Method used like an attribute (getter/setter).
`@classmethod @staticmethod`	Receives `cls` / nothing; alt. constructors, helpers.
`super().__init__()`	Call the parent implementation.
`_x __x`	Convention: internal / name-mangled.
`class C(A, B)`	Multiple inheritance; order = `C.__mro__`.

Exceptions in depth

`raise ValueError('msg')`	Throw an exception.
`raise NewErr('x') from e`	Chain: keep the original cause.
`except (A, B) as e:`	Catch several types.
`class AppError(Exception): pass`	Custom exception hierarchy.
`except* / ExceptionGroup`	Handle grouped errors (3.11+).
`with suppress(KeyError):`	Ignore an error (`contextlib`).

Regex & dates

`re.search(r'\d+', s)`	First match or None.
`re.findall / re.sub / re.compile`	All matches / replace / precompile.
`m.group(1) (?P<name>...)`	Capture groups (numbered / named).
`datetime.now()`	Current local date-time.
`dt.strftime('%Y-%m-%d')`	Format date -> str.
`datetime.strptime(s, fmt)`	Parse str -> date.
`timedelta(days=1)`	Date arithmetic.

LEVEL 3 · ADVANCED

Use Python's data model: protocols, decorators, types and the standard library.

Iterators & generators

```
def count(n):
    i = 0
    while i < n:
        yield i              # pause here, resume on next()
        i += 1

gen = count(3)
next(gen)                    # 0
yield from other_iterable    # delegate to a sub-iterator
```

`iter(x) next(it, default)`	Manual iteration.
`__iter__ / __next__`	Iterator protocol; raise `StopIteration` at the end.
`(x for x in it)`	Generator expression: lazy, O(1) memory.

Decorators & closures

```
import functools, time

def timer(fn):
    @functools.wraps(fn)             # keep name/docstring
    def wrapper(*a, **kw):
        t = time.perf_counter()
        try:
            return fn(*a, **kw)
        finally:
            print(time.perf_counter() - t)
    return wrapper

@timer
def work(): ...                      # work = timer(work)
```

A decorator with arguments is a function that returns a decorator. Closures capture variables, not values (watch loop variables).

Context managers

```
from contextlib import contextmanager
import os

@contextmanager
def cd(path):
    old = os.getcwd()
    os.chdir(path)
    try:
        yield                        # `with` body runs here
    finally:
        os.chdir(old)

# class form: __enter__(self) / __exit__(self, et, ev, tb)
```

Special (dunder) methods

`__init__ __new__`	Initialise / create the instance.
`__repr__ __str__`	Debug string / user-friendly string.
`__eq__ __lt__ __hash__`	Comparison & hashing (define eq and hash together).
`__len__ __bool__ __contains__`	len(), truthiness, `in`.
`__getitem__ __setitem__`	Indexing `obj[k]`.
`__iter__ __next__`	Iteration.
`__call__`	Make an instance callable: `obj(...)`.
`__enter__ __exit__`	Context manager protocol.
`__add__ __mul__ __radd__`	Operator overloading.
`__getattr__ __setattr__`	Attribute fallback / interception.
`__slots__ = ('x', 'y')`	Fixed attributes; less memory, faster.
`__get__ __set__ __set_name__`	Descriptor protocol.
`__init_subclass__`	Hook run when a class is subclassed.

Dataclasses & type hints

```
from dataclasses import dataclass, field

@dataclass(slots=True, frozen=True)
class Point:
    x: float
    y: float = 0.0
    tags: tuple[str, ...] = ()

# mutable default -> field(default_factory=list)
```

`list[int] dict[str, int]`	Generic built-ins (3.9+).
`int | None`	Union syntax (3.10+); replaces Optional.
`Callable[[int], str]`	Function type.
`TypeVar Generic Protocol`	Generics & structural typing.
`Literal Final TypedDict`	Exact values, constants, dict shapes.
`def first[T](xs: list[T]) -> T:`	Inline generics (3.12+).
`mypy pyright`	Static checkers; hints are not enforced at runtime.

Pattern matching (3.10+)

```
match command:
    case ["go", direction]:          # sequence pattern
        move(direction)
    case {"type": "click", "x": x}:  # mapping pattern
        click(x)
    case Point(x=0, y=0):            # class pattern
        origin()
    case int() | float():            # OR pattern
        number()
    case _:                          # default
        unknown()
```

Standard-library power tools

`Counter defaultdict deque`	Counting, auto-default dict, fast double-ended queue.
`namedtuple ChainMap`	Light records; layered dicts.
`chain product combinations`	itertools: join, cartesian product, combos.
`groupby islice accumulate`	Group sorted runs, slice iterators, running totals.
`partial reduce lru_cache`	functools: pre-fill args, fold, memoise.
`cached_property singledispatch`	Cache a value; overload by argument type.
`Path('a') / 'b.txt'`	pathlib: object-oriented paths.
`p.read_text() p.glob('*.py')`	Read file; pattern search.
`Enum auto() StrEnum`	Named constants.
`ABC @abstractmethod`	Interfaces / abstract base classes.
`copy.copy copy.deepcopy`	Shallow / recursive copy.
`heapq bisect`	Priority queue; binary search.

LEVEL 4 · PROFESSIONAL

Ship reliable software: concurrency, tests, tooling, performance, security.

Concurrency model

`threading ThreadPoolExecutor`	I/O-bound work; the GIL blocks CPU parallelism.
`multiprocessing ProcessPoolExecutor`	CPU-bound work in separate processes.
`asyncio`	Cooperative single-thread I/O; thousands of tasks.
`Lock Queue Semaphore`	Synchronise / communicate safely.
`GIL`	One thread runs bytecode at a time (optional free-threaded build since 3.13).

```
import asyncio

async def fetch(n):
    await asyncio.sleep(1)           # non-blocking wait
    return n

async def main():
    a, b = await asyncio.gather(fetch(1), fetch(2))
    async with asyncio.TaskGroup() as tg:     # 3.11+
        t = tg.create_task(fetch(3))
    print(a, b, t.result())

asyncio.run(main())
```

`asyncio.to_thread(fn)`	Run blocking code without freezing the loop.
`asyncio.timeout(5)`	Deadline for a block (3.11+).
`async for / async with`	Async iterators & context managers.

Testing with pytest

```
# test_calc.py        run:  pytest -q
import pytest

def test_add():
    assert add(2, 3) == 5

@pytest.mark.parametrize("a,b,r", [(1, 1, 2), (0, 5, 5)])
def test_many(a, b, r):
    assert add(a, b) == r

def test_error():
    with pytest.raises(ZeroDivisionError):
        div(1, 0)

@pytest.fixture
def client(): ...
```

Also: `pytest-cov` *(coverage),* `unittest.mock` *(isolate dependencies),* `hypothesis` *(property-based tests),* `tox`/`nox` *(test matrix).*

Project tooling

`python -m venv .venv`	Isolated environment per project.
`pip install -r requirements.txt`	Install deps; `pip freeze` to pin.
`uv poetry pipx`	Modern dependency / project / tool managers.
`pyproject.toml`	One config file: metadata, deps, tools.
`ruff black isort`	Lint and format automatically.
`mypy pyright`	Type-check in CI.
`pre-commit`	Run checks before each commit.
`python -m build ; twine upload`	Build & publish a package.

```
[project]
name = "myapp"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = ["httpx>=0.27"]

[tool.ruff]
line-length = 100

[tool.pytest.ini_options]
addopts = "-q"
```

Logging

```
import logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

log.info("started %s", name)   # lazy %-formatting
log.exception("failed")        # logs the traceback too
```

Performance

`python -m cProfile -s cumtime app.py`	Find hot spots: profile before optimising.
`timeit time.perf_counter()`	Micro-benchmarks / timing.
`tracemalloc`	Memory usage and leaks.
`py-spy top --pid N`	Sampling profiler for live processes.

`set / dict lookup`	O(1) vs list O(n).
`''.join(parts)`	Build strings fast (not `+=` in a loop).
`generators, itertools`	Stream large data.
`numpy polars Cython`	Vectorise or compile hot loops.
`functools.cache`	Memoise pure functions.

Internals worth knowing

`Reference counting + GC`	Freed at refcount 0; a cycle collector backs it up.
`weakref`	Reference an object without keeping it alive.
`dis.dis(fn)`	Show bytecode.
`sys.getsizeof(x)`	Shallow size in bytes.
`sys.modules importlib`	Import cache and dynamic imports.
`type(name, bases, ns)`	Classes are objects created by `type` (a metaclass).

```
class Positive:                       # descriptor
    def __set_name__(self, owner, name):
        self.n = "_" + name
    def __get__(self, obj, owner=None):
        return getattr(obj, self.n)
    def __set__(self, obj, v):
        if v <= 0:
            raise ValueError("must be > 0")
        setattr(obj, self.n, v)

class Meta(type):                     # metaclass
    def __new__(mcls, name, bases, ns):
        return super().__new__(mcls, name, bases, ns)
```

Design & best practices

`PEP 8 PEP 20`	Style guide / Zen of Python (`import this`).
`EAFP over LBYL`	Try it and handle the error, don't pre-check everything.
`Composition > inheritance`	Prefer small collaborating objects.
`Type hints on public APIs`	Document intent and enable tooling.
`Docstrings`	Google / NumPy / reST style.
`pydantic`	Runtime validation from type hints.
`Dependency injection`	Pass collaborators in: easy to test.
`12-factor config`	Env vars, never hard-coded secrets.

Security basics

`secrets`	Secure random tokens (not `random`).
`eval exec pickle`	Never use on untrusted input.
`cur.execute(q, params)`	Parameterised SQL prevents injection.
`pip-audit bandit`	Scan dependencies and code for vulnerabilities.
`subprocess.run([...], check=True)`	Pass a list; avoid `shell=True`.

Ecosystem map

`Web`	Django, Flask, FastAPI
`HTTP`	requests, httpx, aiohttp
`Data`	NumPy, pandas, polars
`ML / AI`	scikit-learn, PyTorch, Transformers
`Databases`	SQLAlchemy, sqlite3, psycopg, Alembic
`CLI`	argparse, click, typer
`Background jobs`	Celery, RQ, APScheduler
`Scraping / automation`	BeautifulSoup, Playwright, Selenium

BONUS · GOTCHAS & COMPLEXITY

Mistakes everyone makes once, and the cost of common operations.

Common gotchas

`def f(x=[]):`	Mutable default is shared between calls: use `None`.
`[lambda: i for i in r]`	Late binding: all see the last `i`; use `lambda i=i:`.
`a == b vs a is b`	Equal value vs the same object.
`for x in lst: lst.remove(x)`	Mutating while iterating skips items; iterate a copy.
`b = a (a is a list)`	Alias, not a copy; use `a.copy()` / `deepcopy`.
`0.1 + 0.2 != 0.3`	Floating point: use `math.isclose` or `Decimal`.
`except:`	Bare except hides bugs; catch specific errors.
`from x import *`	Pollutes the namespace; import explicitly.
`10 / 2 gives 5.0`	`/` always returns float; use `//` for ints.
`'a' + 1`	TypeError: Python never coerces str and int.

Big-O of common operations

`list[i] append pop()`	O(1)
`list.insert / remove / x in list`	O(n)
`dict / set: get, set, in`	O(1) average
`sorted / list.sort`	O(n log n) (Timsort, stable)
`deque append / pop (both ends)`	O(1)
`heapq push / pop`	O(log n)
`bisect on a sorted list`	O(log n)
`s += x in a loop`	O(n^2): use `''.join()`

Practice projects by level

`Beginner`	Number-guessing game, unit converter, CLI to-do list, text-adventure.
`Intermediate`	File organiser, CSV/JSON report generator, web scraper, quiz app with classes.
`Advanced`	Plugin system with decorators, LRU cache, pipeline of generators, typed library.
`Professional`	FastAPI service with tests, CI, Docker; async crawler; packaged CLI on PyPI.